

NAIL094 — Decision Procedures and SAT/SMT Solvers

DPLL Experimental Evaluation

Zerina Ahmetovic

1 Task overview

The implemented DPLL solver with basic branch and bound DPLL algorithm and unit propagation is tested on small benchmarks. The correct input format is either SMT-LIB or DIMACS, and output writes out whether the formula is SAT (in which case a model is shown as well) or UNSAT, along with statistical information (CPU time, number of decisions and steps of unit propagation).

Two sets of benchmarks are used. The first is the five example formulas given with the preceding task of the course [2], which asks for a Tseitin encoding of NNF formulas written in a simplified SMT-LIB syntax (Table 1). The second is three families from SATLIB [3]: All Interval Series [4], pigeonhole [5] and Uniform Random-3-SAT [6], collected in Table 2. The first two are structured and are the families suggested for this task; `ais6`, `ais8`, `ais10`, `hole6` and `hole7` are all included, together with the larger `ais12`, `hole8` and `hole9` so that the growth in solving time is visible over a wider range. The random instances are included for contrast only, being uniformly generated rather than built from a combinatorial principle.

2 Implementation

The solver is invoked as `dp11 [input]`. The input format follows the file extension, `.cnf` for DIMACS and `.sat` for the simplified SMT-LIB format, and can be forced with `-cnf` or `-sat`. SMT-LIB input is passed through the Tseitin encoder written for the preceding task [2], so the solver itself only ever sees CNF.

The algorithm consists of three fundamental parts, explained below.

Trail: The current partial assignment, kept three ways at once: a value per literal so that testing whether a literal is false is a single index flip, a stack in assignment order so that undoing runs in reverse, and a mark per decision level recording where that level began. The decision literal of level $d + 1$ is the first entry after mark d , which is how a decision is recovered when it has to be flipped. The stack doubles as the propagation queue.

Unit propagation: For each literal l , `occ[1]` lists the clauses containing it. Assigning l true can only shorten clauses containing $\neg l$, so only those are visited; the work per assignment is proportional to the occurrences of the assigned literal rather than to the size of the formula.

Search: Branch on the lowest-indexed unassigned variable, positive first, which the task permits. On a conflict the solver walks up to the deepest decision level that has not yet been tried in both polarities, discards everything below it and re-decides the negated literal; if no such level remains the formula is unsatisfiable. One bit per decision level replaces the recursion, so the search is a flat loop.

Clauses are normalised once at load: repeated literals collapsed, tautological clauses dropped, and clauses repeating an earlier one dropped. The last matters on All Interval Series, where `ais6` loses 140 of its 581 clauses as duplicates, which is why Table 2 reports 441 clauses for it rather than the 581 of the original file.

3 Experimental evaluation

The times in Tables 1 and 2 were measured with g++ 14.2.0 (MSYS2 UCRT64) at `-std=c++20 -O2`. Each instance was run three times and the fastest kept, the timer covers the search only, reading the file or encoding it are not included.

Example formulas in SMT-LIB format

instance	vars	clauses	result	decisions	unit props	CPU s
toy_5.sat	9	13	SAT	5	3	0.000
toy_50.sat	99	148	SAT	50	48	0.000
toy_100.sat	199	298	SAT	100	98	0.000
nested_5.sat	22	45	SAT	6	15	0.000
nested_8.sat	136	371	SAT	65	1 596	0.000

Table 1: The five example inputs from the first task, read in the simplified SMT-LIB format and encoded by Tseitin before solving. Variable and clause counts are those of the resulting CNF.

As Table 1 shows, all five examples are satisfiable, and only `nested_8` produces any conflicts at all (31). The other four are solved without backtracking once, so the Tseitin encoding of a satisfiable NNF formula leaves the solver very little to do.

SATLIB benchmarks

instance	vars	clauses	result	decisions	unit props	CPU s
hole6	42	133	UNSAT	6 490	29 322	0.001
hole7	56	204	UNSAT	65 560	318 495	0.009
hole8	72	297	UNSAT	756 686	3 933 538	0.124
hole9	90	415	UNSAT	9 825 028	54 417 935	1.635
ais6	61	441	SAT	57	515	0.000
ais8	113	1149	SAT	550	7 423	0.001
ais10	181	2377	SAT	8 973	138 777	0.017
ais12	265	4269	SAT	192 780	3 389 633	0.355
uf50-01	50	218	SAT	1 016	7 573	0.001
uf50-010	50	218	SAT	370	2 526	0.001
uf50-0100	50	218	SAT	318	2 355	0.000
uf50-0101	50	218	SAT	44	224	0.000

Table 2: SATLIB instances from three categories. CPU time is the fastest of three runs and covers search only, not parsing. Clause counts are after duplicate and tautological clauses are removed at load.

Table 2 shows that inside the pigeonhole family the decision count grows by factors of 10.1, 11.5 and 13.0 from `hole6` to `hole9`, so the growth factor itself increases rather than staying constant.

CPU time follows it, from 0.001 s to 1.635 s for 48 additional variables. A running time of 2^{cn} would give a constant factor; one rising by about 1.4 per step is the shape of $n!$ -type growth, that is $2^{\Theta(n \log n)}$. That is the expected shape here, since DPLL without clause learning corresponds to tree-like resolution, for which the pigeonhole principle is known to require refutations of that size.

Size alone does not predict difficulty. Across families in the same table, `ais12` has 265 variables against `hole9`'s 90 and is still solved in 0.355 s rather than 1.635 s, being satisfiable and far less constrained. Within a family the same holds: the four `uf50` instances are identical in size yet differ by a factor of 23 in decisions. These sit at the phase-transition ratio $218/50 = 4.36$, where difficulty is heavy-tailed even at fixed size, so with a fixed index-order decision rule how soon the search meets a solution varies enormously from one instance to the next. Size alone predicts neither.

The task also asks for the number of unit propagation steps, which Table 2 reports. Read per decision it separates the families: pigeonhole stays near 4.5 to 5.5 and `uf50` near 5 to 7.5, while All Interval Series climbs from 9.0 on `ais6` to 17.6 on `ais12`. Each decision there forces far more assignments, and increasingly so with size, which is the implication structure the later look-ahead solver exploits.

Code

The full source is in the accompanying repository [1]. `src/dpll.hpp` holds the trail, the statistics, the adjacency-list propagation and the search loop; `src/dpll.cpp` is the command line, format dispatch and output. The DIMACS reader is `src/cnf.hpp` and the SMT-LIB path reuses `src/formula.hpp` and `src/tseitin.hpp`, written for the encoding task [2]. Correctness is checked by `tests/check_solver.py`, which compares the verdict against exhaustive search and validates the reported model on both input formats, and by `tests/fuzz.py` over random instances. Measurements come from `tests/bench_dpll.py`.

Summary

The solver reads both required formats, reports SAT or UNSAT with a model, and prints the three required statistics. Solving time grows steeply with instance size within a family, most sharply on the unsatisfiable pigeonhole instances, but the comparison across families in Table 2 shows that structure and satisfiability matter more than size.

References

- [1] Z. Ahmetovic. Source repository for the NAIL094 programming tasks. <https://github.com/inferno73/SAT-solver-NAIL094>
- [2] P. Kučera. Tseitin encoding and DIMACS format, programming task for NAIL094. https://ktiml.mff.cuni.cz/~kucerap/satsmt/practical/task_tseitin.php
- [3] H. H. Hoos and T. Stützle. SATLIB: An Online Resource for Research on SAT. <https://www.cs.ubc.ca/~hoos/SATLIB/benchm.html>
- [4] SATLIB benchmark problems, All Interval Series. <https://www.cs.ubc.ca/~hoos/SATLIB/Benchmarks/SAT/AIS/descr.html>
- [5] SATLIB benchmark problems, DIMACS pigeon hole. <https://www.cs.ubc.ca/~hoos/SATLIB/Benchmarks/SAT/DIMACS/PHOLE/descr.html>

- [6] SATLIB benchmark problems, Uniform Random-3-SAT. <https://www.cs.ubc.ca/~hoos/SATLIB/Benchmarks/SAT/RND3SAT/descr.html>